

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 1&2

Student Name: N.Samhavva

Roll No.: A24126552099

Date: 29-08-2026

1. TITLE

Design Static Webpage using HTML Components

2. AIM

To design and develop a structured static webpage using **HTML5 semantic elements** without using CSS, demonstrating document structure, navigation, articles, forms, lists, tables, and native HTML components.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the basic structure of an HTML5 document.
 - To use semantic HTML elements for organizing webpage content.
 - To create navigation links within a webpage.
 - To structure content using `<header>`, `<main>`, `<section>`, `<article>`, `<aside>`, and `<footer>`.
 - To create and organize a webpage layout using HTML tables.
 - To implement native HTML form controls.
 - To understand HTML input validation using attributes such as `required`.
 - To display ordered and unordered lists.
 - To develop a functional static webpage without using CSS.
-

4. THEORY

HTML (HyperText Markup Language) is the standard markup language used to create and structure content on webpages. HTML provides various elements for defining headings, paragraphs, links, forms, lists, tables, and other webpage components.

Semantic HTML uses meaningful elements that clearly describe the purpose and structure of the content. Examples include `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, and `<footer>`.

In this experiment, a static webpage named **StaticWeb** is created using pure HTML without external or internal CSS. The browser's default rendering is used to display the webpage.

The webpage consists of:

- A **header** containing the website title and navigation menu.
- A **home section** containing the main heading and introduction.
- A **main content area** containing two articles.
- An **interactive form** for collecting user information.
- A **sidebar** containing references, benefits, and required components.
- A **footer** containing copyright information.

The basic structure of the webpage is:

HTML Document → Header → Navigation → Main Content → Articles → Form → Sidebar → Footer

The form uses different native HTML controls such as text input, email input, dropdown selection, textarea, submit button, and reset button.

5. ALGORITHM / PROCEDURE

1. Create a new HTML file with the `.html` extension.
2. Declare the HTML5 document type using `<!DOCTYPE html>`.
3. Create the `<html>` element and specify the language as English.
4. Add the `<head>` section containing character encoding and webpage title.
5. Create a `<header>` section for the website name and navigation menu.
6. Add navigation links using the `<nav>` and `<a>` elements.
7. Create a home/showcase section using the `<section>` element.
8. Add headings, paragraphs, and internal navigation links.
9. Create the main webpage content using `<main>`.
10. Organize individual topics using `<article>` elements.
11. Create an interactive form using `<form>` and `<fieldset>`.
12. Add text, email, select, textarea, submit, and reset controls.
13. Create a sidebar using the `<aside>` element.
14. Add unordered and ordered lists to the sidebar.
15. Create a `<footer>` section containing copyright information.
16. Save the HTML file.
17. Open the file in a web browser.
18. Verify the webpage structure and navigation links.
19. Test the form fields and browser-level validation.
20. Record the actual output and verify it with the expected output.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Pure Semantic HTML Static Webpage</title>
</head>
<body>
  <!-- Header Section -->
  <header>
    <table width="100%" border="0" cellspacing="0" cellpadding="10">
      <tr>
        <td>
          <h1>StaticWeb</h1>
        </td>
        <td align="right">
          <nav>
            <a href="#home">Home</a> &nbsp;|&nbsp;&nbsp;
            <a href="#articles">Articles</a> &nbsp;|&nbsp;&nbsp;
            <a href="#about">About</a> &nbsp;|&nbsp;&nbsp;
            <a href="#contact">Contact Us</a>
          </nav>
        </td>
      </tr>
    </table>
  </header>
  <hr>
  <!-- Hero / Showcase Section -->
  <section id="home">
    <center>
      <h2>Semantic Structures Without CSS</h2>
      <p>This layout uses raw HTML components to build an organized document hierarchy
that browsers can cleanly parse out-of-the-box.</p>
      <p><a href="#articles"><b>Read Our Articles Below &darr;</b></a></p>
    </center>
  </section>
  <hr>
  <!-- Main Content Layout using a Structural Table -->
  <table width="100%" border="0" cellspacing="0" cellpadding="20">
    <tr valign="top">
      <!-- Left Side: Main Content Column -->
      <td width="70%">
        <main id="articles">
          <article>
```

1. Core Structural Tags

Published: July 7, 2026

When you omit cascading style sheets entirely, the semantic markup handles all of the structural heavy lifting. Elements like `<main>`, `<article>`, and `<section>` tell screen readers and web scrapers exactly how information flows.

Using raw browser defaults ensures excellent loading performance, perfect accessibility compliance, and absolute structural clarity.

`</article>`

`<br><hr><br>`

`<article>`

2. Native Interaction Elements

Published: July 5, 2026

Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.

Interactive Demonstration Forms

`<form id="contact" action="#" method="post">`

`<fieldset>`

`<legend><b>User Inquiry Form</b></legend>`

`<p>`

`<label for="name">Full Name: </label>`

`<input type="text" id="name" name="username" required`

`placeholder="John Doe">`

`</p>`

`<p>`

`<label for="email">Email Address: </label>`

`<input type="email" id="email" name="useremail" required>`

`</p>`

`<p>`

`<label for="topic">Inquiry Topic: </label>`

`<select id="topic" name="usertopic">`

`<option value="general">General Feedback</option>`

`<option value="technical">Technical Architecture</option>`

`<option value="academics">Academic Curriculum</option>`

`</select>`

`</p>`

`<p>`

`<label for="message">Message Details:</label><br>`

`<textarea id="message" name="usermessage" rows="5" cols="40"`

`placeholder="Type your notes here..."></textarea>`

`</p>`

`<p>`

`<input type="submit" value="Submit Form Data">`

`<input type="reset" value="Clear Entries">`

`</p>`

`</fieldset>`

```

        </form>
    </article>
</main>
</td>
<!-- Right Side: Sidebar Column -->
<td width="30%" bgcolor="#f4f4f4">
    <aside id="about">
        <h3>Document References</h3>
        <p>Static pages map data out directly without real-time database queries or
processor overhead.</p>
        <h4>Core Benefits:</h4>
        <ul>
            <li>Instant browser painting</li>
            <li>Low memory footprints</li>
            <li>High data compatibility</li>
        </ul>
        <h4>Required Components:</h4>
        <ol>
            <li>Document Type Definition</li>
            <li>Semantic Wrapper Blocks</li>
            <li>Data Input Control Interfaces</li>
        </ol>
    </aside>
</td>
</tr>
</table>
<hr>
<!-- Footer Section -->
<footer>
    <center>
        <p>&copy; 2026 Pure HTML Architecture. Assembled completely without style
configurations.</p>
    </center>
</footer>
</body>
</html>

```

7. CODE EXPLANATION

HTML Code / Element	Explanation
<!DOCTYPE html>	Declares that the document uses HTML5.
<html lang="en">	Defines the root HTML element and specifies English as the document language.

HTML Code / Element	Explanation
<head>	Contains metadata and document information.
<meta charset="UTF-8">	Specifies UTF-8 character encoding.
<title>	Defines the title displayed in the browser tab.
<body>	Contains the visible content of the webpage.
<header>	Defines the introductory/header section of the webpage.
<nav>	Contains navigation links to different sections.
<a>	Creates hyperlinks within the webpage.
<section>	Defines a thematic section of the document.
<main>	Represents the main content of the webpage.
<article>	Represents independent content such as an article or post.
<aside>	Defines supplementary or sidebar content.
<footer>	Defines the footer section of the webpage.
<table>	Organizes content into rows and columns.
<h1>– <h4>	Defines headings with different levels of importance.
<p>	Defines a paragraph.
<hr>	Creates a horizontal thematic break.
<form>	Creates a form for collecting user input.
<fieldset>	Groups related form controls together.
<legend>	Provides a caption for the fieldset.
<label>	Provides a descriptive label for a form control.
<input type="text">	Accepts text input from the user.
<input type="email">	Accepts an email address and provides browser validation.
required	Makes the specified form field mandatory.
<select>	Creates a dropdown list.

<option>	Defines an option inside a dropdown list.
<textarea>	Provides a multi-line text input area.
type="submit"	Submits the form data.
type="reset"	Clears the entered form data.
	Creates an unordered/bulleted list.
	Creates an ordered/numbered list.
	Defines an individual list item.
<code>	Displays text as code.
id	Provides a unique identifier used for internal navigation.
href="#..."	Creates navigation to a specific section of the same webpage.
action="#"	Specifies the destination for the form submission; #keeps the action on the current page.
method="post"	Specifies POST as the form submission method.

8. TEST CASES AND EXECUTION DATA

The webpage was tested by opening the HTML file in a web browser and verifying the page structure, navigation, form controls, and browser validation.

Test Case	TC-01
Test / Input	Load the webpage in the browser
Expected Result	Header, navigation bar, main heading, articles, form, document references, and footer should display correctly.
Actual Result	All webpage sections were displayed correctly.
Status	Pass

TC-01 Output:

Semantic Structures Without CSS

This layout uses raw HTML components to build an organized document hierarchy that browsers can cleanly parse out-of-the-box.

[Read Our Articles Below.](#)

1. Core Structural Tags

Published: July 7, 2026

When you omit cascading style sheets entirely, the semantic markup handles all of the structural heavy lifting. Elements like <main>, <article>, and <section> tell screen readers and web scrapers exactly how information flows.

Using raw browser defaults ensures excellent loading performance, perfect accessibility compliance, and absolute structural clarity.

2. Native Interaction Elements

Published: July 5, 2026

Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.

Interactive Demonstration Forms

User Inquiry Form

Full Name:

Email Address:

Inquiry Topic:

General Feedback

Message Details:

Type your notes here...

Submit Form Data

Clear Entries

Document References

Static pages map data out directly without real-time database queries or processor overhead.

Core Benefits:

- Instant browser painting
- Low memory footprints
- High data compatibility

Required Components:

- Document Type Definition
- Semantic Wrapper Blocks
- Data Input Control Interfaces

© 2026 Pure HTML Architecture. Assembled completely without style configurations.

Test Case	TC-02
Test / Input	Check the navigation links: Home, Articles, About, and Contact Us
Expected Result	All navigation links should be visible and clickable.
Actual Result	All navigation links were displayed correctly.
Status	Pass

TC-02 Output:

Test Case	TC-03
Test / Input	Check the main heading and introductory content
Expected Result	"Semantic Structures Without CSS", the description, and "Read Our Articles Below" link should be displayed correctly.
Actual Result	The heading, description, and link were displayed correctly.
Status	Pass

TC-03 Output:

Semantic Structures Without CSS

This layout uses raw HTML components to build an organized document hierarchy that browsers can cleanly parse out-of-the-box.

[Read Our Articles Below.](#)

Test Case	TC-04
Test / Input	Check the article sections and publication dates
Expected Result	"1. Core Structural Tags" and "2. Native Interaction Elements", along with their publication dates and content, should be displayed correctly.
Actual Result	Both article sections and their content were displayed correctly.
Status	Pass

TC-04 Output:

1. Core Structural Tags

Published: July 7, 2026

When you omit cascading style sheets entirely, the semantic markup handles all of the structural heavy lifting. Elements like `<main>`, `<article>`, and `<section>` tell screen readers and web scrapers exactly how information flows.

Using raw browser defaults ensures excellent loading performance, perfect accessibility compliance, and absolute structural clarity.

2. Native Interaction Elements

Published: July 5, 2026

Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.

Interactive Demonstration Forms

User Inquiry Form

Full Name:

Email Address:

Inquiry Topic:

General Feedback

Message Details:

Type your notes here...

Submit Form Data

Clear Entries

Test Case	TC-05
Test / Input	Check the User Inquiry Form
Expected Result	Full Name, Email Address, Inquiry Topic, Message Details, Submit Form Data, and Clear Entries controls should be displayed.
Actual Result	All form fields and buttons were displayed correctly.
Status	Pass

TC-05 Output:

Interactive Demonstration Forms

User Inquiry Form

Full Name:

Email Address:

Inquiry Topic:

General Feedback

Message Details:

Type your notes here...

Submit Form Data

Clear Entries

Test Case	TC-06
Test / Input	Enter sample data into the User Inquiry Form
Expected Result	The user should be able to enter the name, email address, select an inquiry topic, and enter message details.
Actual Result	Sample data was entered successfully into the form fields.
Status	Pass

TC-06 Output:

Interactive Demonstration Forms

User Inquiry Form

Full Name:

Email Address:

Inquiry Topic:

Message Details:

Type your notes here...

Test Case	TC-07
Test / Input	Check the Document References section
Expected Result	The Document References heading, Core Benefits list, and Required Components list should be displayed correctly.
Actual Result	The Document References section and both lists were displayed correctly.
Status	Pass

TC-07 Output:

Document References

Static pages map data out directly without real-time database queries or processor overhead.

Core Benefits:

- Instant browser painting
- Low memory footprints
- High data compatibility

Required Components:

1. Document Type Definition
2. Semantic Wrapper Blocks
3. Data Input Control Interfaces

Test Case	TC-08
Test / Input	Check the webpage footer
Expected Result	The copyright and architecture information should be displayed at the bottom of the webpage.
Actual Result	The footer was displayed correctly.
Status	Pass

TC-08 Output:

9. OUTPUT

StaticWeb

[Home](#) | [Articles](#) | [About](#) | [Contact Us](#)

Semantic Structures Without CSS

This layout uses raw HTML components to build an organized document hierarchy that browsers can cleanly parse out-of-the-box.

[Read Our Articles Below :](#)

1. Core Structural Tags

Published: July 7, 2026

When you omit cascading style sheets entirely, the semantic markup handles all of the structural heavy lifting. Elements like `<main>`, `<article>`, and `<section>` tell screen readers and web scrapers exactly how information flows.

Using raw browser defaults ensures excellent loading performance, perfect accessibility compliance, and absolute structural clarity.

2. Native Interaction Elements

Published: July 5, 2026

Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.

Interactive Demonstration Forms

User Inquiry Form

Full Name:

Email Address:

Inquiry Topic:

General Feedback

Message Details:

Type your notes here,...

Submit Form Data

Clear Entries

Document References

Static pages map data out directly without real-time database queries or processor overhead.

Core Benefits:

- Instant browser painting
- Low memory footprints
- High data compatibility

Required Components:

- Document Type Definition
- Semantic Wrapper Blocks
- Data Input Control Interfaces

© 2026 Pure HTML Architecture. Assembled completely without style configurations.

10. RESULT

Thus, a static webpage was successfully developed using **HTML5 semantic elements without CSS**. The webpage was structured using header, navigation, section, main, article, aside, and footer elements. Internal navigation, lists, tables, and native HTML form controls were implemented successfully. The webpage and form validation were tested using multiple test cases and the expected results were obtained.